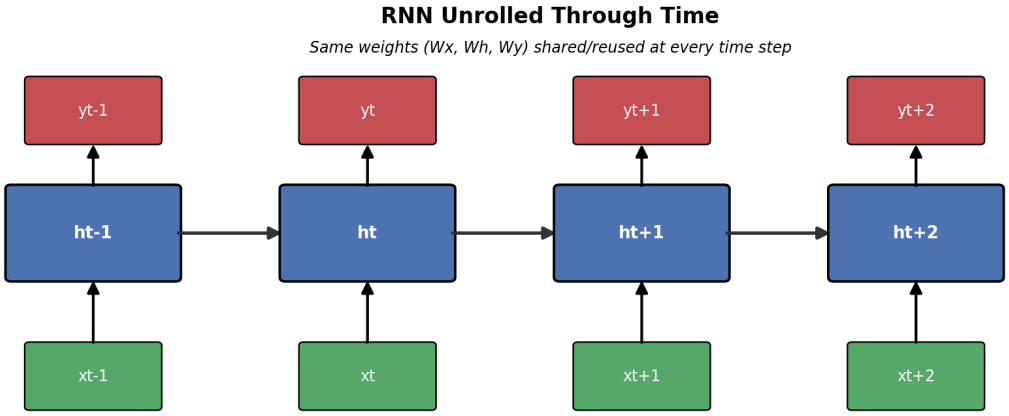
Recurrent Neural Networks (RNN)

A Complete, Visual Guide to Understanding, Building and Training RNNs



Covers: What is an RNN • Why we need RNNs • Types of RNN • How an RNN works • Forward Propagation • Backpropagation Through Time (BPTT) • A worked demo • Applications

1. Why Do We Need RNNs?

Traditional feedforward neural networks (and even standard CNNs) assume every input is **independent** of every other input. Each example goes in, a prediction comes out, and the network has no memory of anything it processed before. That works fine for tasks like classifying a single photo, but it breaks down for **sequential data** - data where order and context matter.

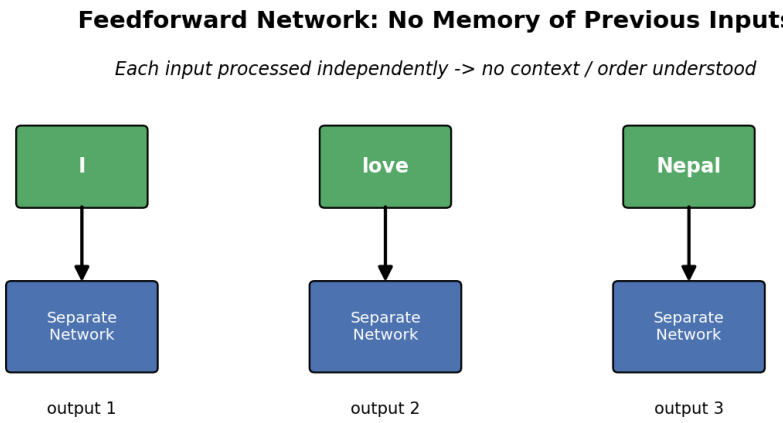


Figure 1: A feedforward network treats each word separately - it cannot connect 'I', 'love', and 'Nepal' into one coherent thought.

Consider these examples where order and memory matter:

- **Language:** "The clouds are in the ___" - predicting 'sky' requires remembering the earlier words.
- **Speech recognition:** understanding a phoneme depends on the sounds that came before it.
- **Stock prices / time-series:** tomorrow's trend depends on the recent past, not just today.
- **Video:** understanding a frame often requires context from previous frames.

RNNs solve this by adding a loop. They carry a running summary of everything seen so far, called the **hidden state**, and update it at every time step. This gives the network a form of memory, allowing it to model sequences of arbitrary length using a single, shared set of weights.

2. What Is a Recurrent Neural Network?

A Recurrent Neural Network is a class of neural network designed for sequential data. Unlike a feedforward network, an RNN's hidden layer feeds its own output back into itself as part of the input for the next time step. This recurrent connection is what allows information to persist across a sequence.

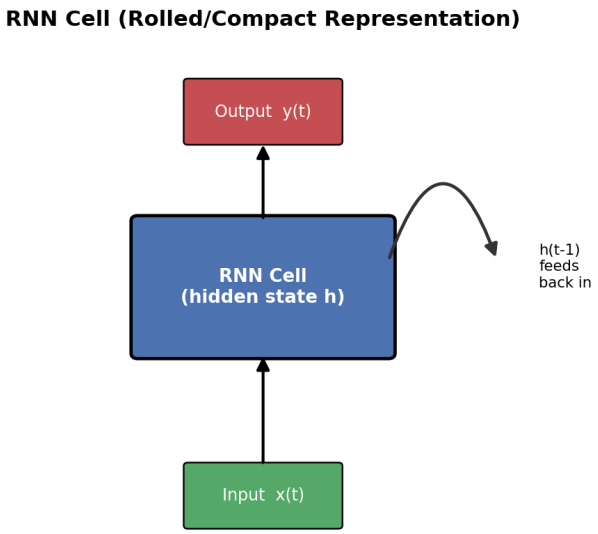


Figure 2: The 'rolled' view of an RNN cell - the hidden state loops back into itself at every step.

When you 'unroll' this loop across time, you get a chain of identical cells, one per time step, all sharing the exact same weights (Wx, Wh, Wy). This weight-sharing is what lets an RNN handle sequences of any length with a fixed number of parameters.

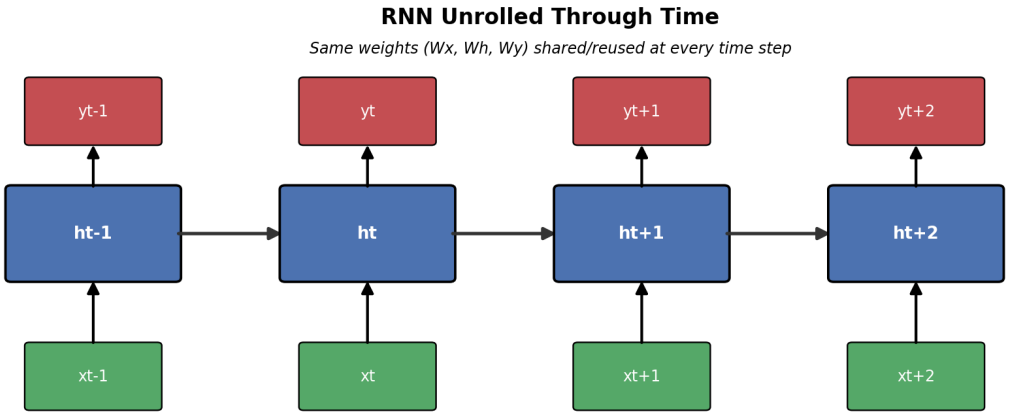


Figure 3: The same RNN cell, unrolled through 4 time steps. Notice the weights are identical at every step.

3. Types of RNN Architectures

RNNs are flexible about how many inputs and outputs they use, which leads to four broad architecture patterns depending on the task:

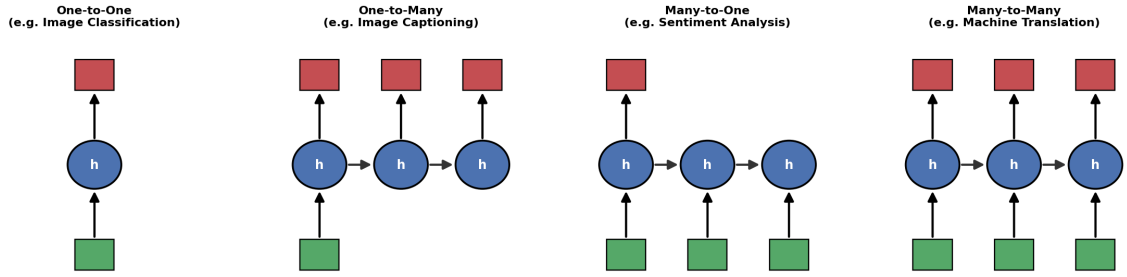


Figure 4: The four canonical RNN input/output patterns.

Type	Structure	Typical Use Case
One-to-One	Single input -> Single output	Standard classification (barely 'recurrent')
One-to-Many	Single input -> Sequence output	Image captioning (image -> sentence)
Many-to-One	Sequence input -> Single output	Sentiment analysis, spam detection
Many-to-Many	Sequence input -> Sequence output	Machine translation, video labeling

There are also several well-known RNN **variants** built to fix specific weaknesses of the basic ("vanilla") RNN:

- **Vanilla RNN** - the basic form described in this guide; struggles with long sequences.
- **LSTM (Long Short-Term Memory)** - adds gates (input, forget, output) and a cell state to preserve long-range dependencies.
- **GRU (Gated Recurrent Unit)** - a simplified, faster variant of LSTM with fewer gates.
- **Bidirectional RNN** - processes the sequence forward AND backward, combining both contexts.
- **Deep (stacked) RNN** - stacks multiple recurrent layers to learn richer representations.

4. How an RNN Works - Forward Propagation

At every time step *t*, the RNN cell takes two inputs: the current input x(t), and the previous hidden state h(t-1). It combines them with a set of weights to produce a new hidden state h(t), which is then optionally passed through an output layer to produce a prediction y(t).

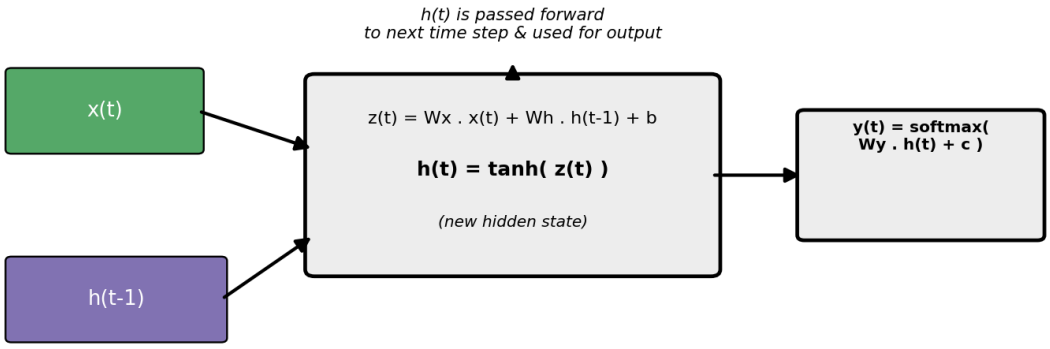


Figure 5: The forward pass inside a single RNN cell at time step t.

The core equations are:

$$h(t) = \tanh(Wx \cdot x(t) + Wh \cdot h(t-1) + b)$$

$$y(t) = \text{softmax}(Wy \cdot h(t) + c)$$

Where:

- **x(t)** - input vector at time step t
- **h(t-1)** - hidden state carried over from the previous time step (the 'memory')
- **Wx, Wh, Wy** - weight matrices, shared across every time step
- **b, c** - bias vectors
- **tanh** - non-linear activation (sigmoid or ReLU variants are also used)
- **softmax** - converts the output layer into a probability distribution (for classification tasks)

This process repeats at every time step, each time reusing the exact same weights, with h(t) carrying forward a compressed summary of everything the network has seen up to that point.

5. Worked Demo: Forward Pass by Hand

Let's trace a tiny numeric example so the mechanics are concrete. Suppose we feed the 3-word sequence **"I", "love", "AI"** into a toy RNN with a 1-dimensional hidden state, starting from h(0) = 0.

Time step	Input x(t)	Computation	New hidden state h(t)
t=1 ("I")	x1 = 0.5	tanh(0.5*0.9 + 0*0.8 + 0)	h1 = tanh(0.45) = 0.42
t=2 ("love")	x2 = 0.8	tanh(0.8*0.9 + 0.42*0.8 + 0)	h2 = tanh(1.056) = 0.78
t=3 ("AI")	x3 = 0.3	tanh(0.3*0.9 + 0.78*0.8 + 0)	h3 = tanh(0.894) = 0.71

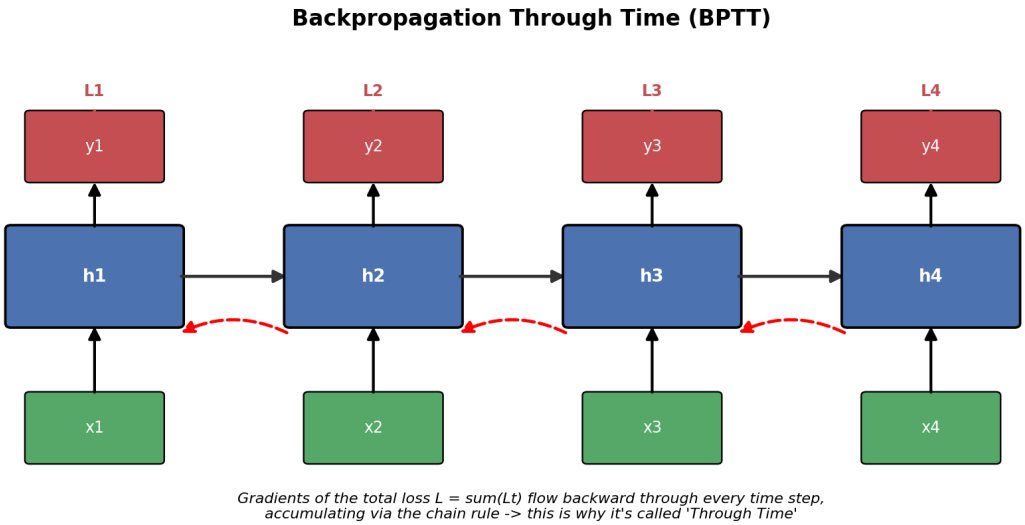
(Using illustrative weights Wx = 0.9, Wh = 0.8, b = 0.) Notice how h3 - the final hidden state - carries a blended trace of all three words, weighted more heavily toward the most recent input. This final h3 could now be passed to an output layer to, say, classify the sentence's sentiment.

A quick mental model:

- Think of h(t) as a notebook the network keeps updating as it reads a sentence word by word.
- At each word, it doesn't erase the notebook - it blends the new word in with what's already written.
- By the end of the sentence, the notebook holds a compressed summary of everything read so far.

6. How Backpropagation Works in RNN - BPTT

Training an RNN means adjusting Wx, Wh, and Wy so that predictions y(t) get closer to the true targets. Because the same weights are reused at every time step, we can't backpropagate normally - we need a special version called **Backpropagation Through Time (BPTT)**.



Gradients of the total loss L = sum(Lt) flow backward through every time step, accumulating via the chain rule -> this is why it's called 'Through Time'

Figure 6: BPTT - the error at every time step is backpropagated through the unrolled network, all the way to earlier steps.

The idea, step by step:

- **1. Unroll the network** across all T time steps, treating it like one long feedforward network.
- **2. Compute the loss** at each time step, Lt (e.g. cross-entropy between y(t) and the true label), then sum them: L = L1 + L2 + ... + LT.
- **3. Backpropagate the total loss** from the last time step back to the first, applying the chain rule at every step.
- **4. Accumulate gradients** for the shared weights Wx, Wh, Wy across all time steps, since the same weight was used repeatedly.
- **5. Update the weights** once, using the summed gradient (e.g. via gradient descent / Adam).

The key gradient equation (conceptually):

$$dL/dWh = \Sigma_t \, dLt/dWh \quad \text{(summed contribution from every time step)}$$

Because h(t) depends on h(t-1), which depends on h(t-2), and so on, the gradient with respect to an early hidden state has to pass through a long chain of multiplications (one per time step, via the chain rule). This is the defining trait of BPTT: **error signals literally travel backward through time**, not just backward through layers.

The Vanishing / Exploding Gradient Problem

Because the same weight matrix Wh is multiplied repeatedly along this backward chain, the gradient can shrink toward zero (vanish) or grow uncontrollably (explode) as it travels back through many time steps - especially with tanh/sigmoid activations. This makes it very hard for vanilla RNNs to learn dependencies that span long sequences.

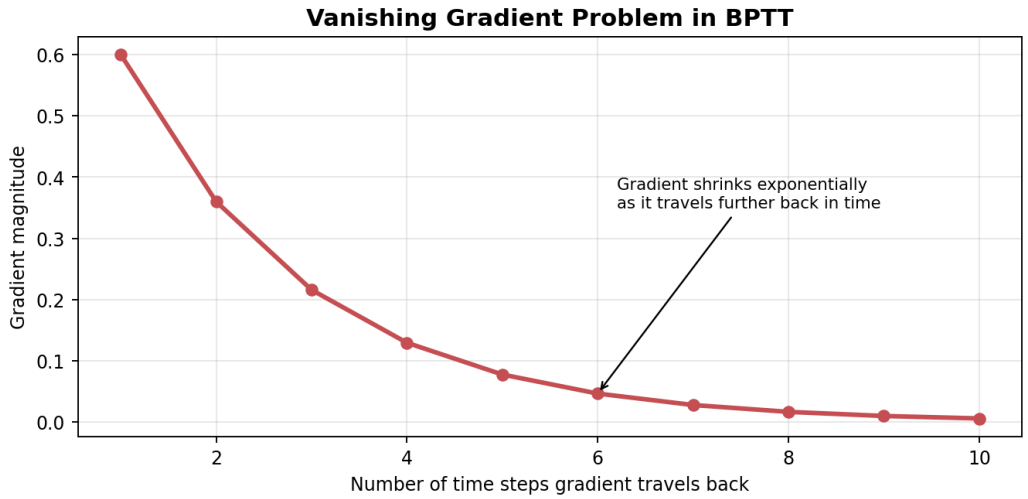


Figure 7: Gradient magnitude shrinking exponentially the further back in time it must travel.

This is precisely the problem that LSTM and GRU cells were designed to fix, using gating mechanisms that let gradients flow through many time steps largely unchanged.

In practice, two common fixes are used alongside gating:

- **Gradient clipping** - capping the gradient's magnitude to prevent exploding gradients.
- **Truncated BPTT** - backpropagating only a limited number of steps back, instead of the entire sequence, for efficiency and stability.

7. Applications of RNNs

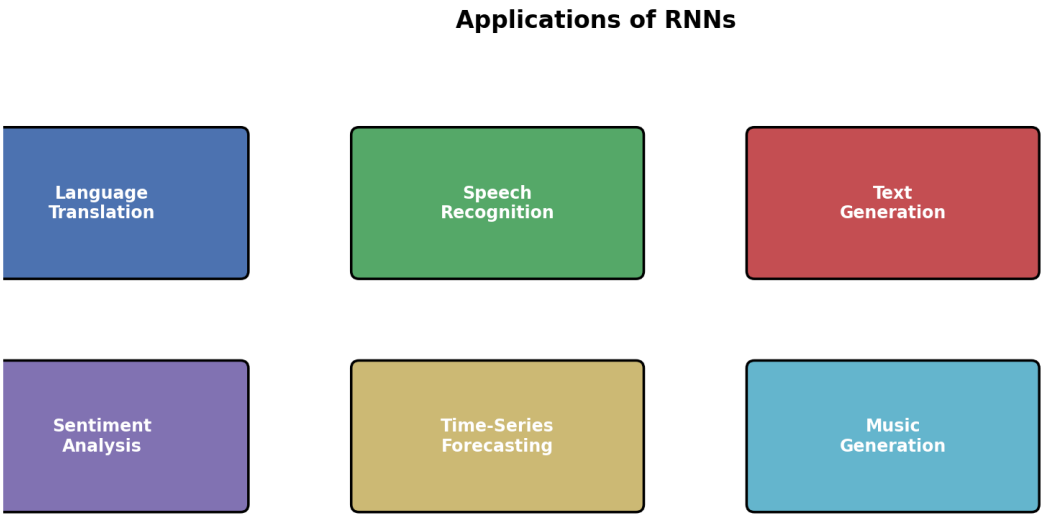


Figure 8: Common real-world applications of RNNs.

- **Natural Language Processing:** machine translation, text generation, chatbots, next-word prediction.
- **Speech Recognition:** converting spoken audio into text (e.g. voice assistants).
- **Sentiment Analysis:** classifying the emotional tone of reviews, tweets, or messages.
- **Time-Series Forecasting:** stock price prediction, weather forecasting, demand forecasting.
- **Music & Text Generation:** generating new sequences that mimic a learned style.
- **Video Analysis:** action recognition and captioning by processing frame sequences.
- **Anomaly Detection:** spotting unusual patterns in sequential sensor or log data.

8. Summary

Concept	Key Idea
Why RNN	Needed for sequential data where order and context matter
Core mechanism	A hidden state h(t) that loops back and carries memory forward
Forward pass	h(t) = tanh(Wx.x(t) + Wh.h(t-1) + b); y(t) = softmax(Wy.h(t) + c)
Types	One-to-one, one-to-many, many-to-one, many-to-many
Training	Backpropagation Through Time (BPTT) - unroll, then backprop the summed loss

Main weakness	Vanishing / exploding gradients over long sequences
Common fixes	LSTM, GRU, gradient clipping, truncated BPTT

With this foundation, the natural next step is exploring LSTM and GRU architectures, which build directly on everything covered here to solve the long-range dependency problem.